OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TECHNOLOGICAL UNIVERSITY DUBLIN

SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

FORMAL ELEMENT

INDIVIDUAL REPORT

by

NAME
[Cathal Flood]

[8/12/24]

Submission Checklist and Declaration

To ensure that the focus of the assessment of your laboratory report can include the development of higher order skills and competences associated with a Level 7 qualification, please complete the checklist and declaration below.

I declare that the report that I am submitting:

▪ is my original work, with secondary sources acknowledged;	Y
▪ was proofread thoroughly for typographical errors;	y
▪ has all red guideline text removed;	y
▪ has all pages numbered in the body of the report;	y
▪ has all diagrams and code listings labeled and numbered;	y
▪ has not exceeded the word count in the body of the report;	Y
▪ contains citations with references formatted in either the Harvard or IEEE citation style.	Y
I understand that my work can be returned uncorrected if the criteria are unfulfilled.	Y

Signature:

Cathal Flood

Contents

Introduction 1
Design 2
High Level Design: Structure Chart: 2
Low Level Design: Algorithm: 5
Implementing the Program 7
Testing 10
Analysis 12
Conclusions 14

TABLE OF FIGURES

Figure 1: The Problem 1
Figure 2: Program block diagram ..... **Error! Bookmark not defined.**
Figure 2: Program Flowchart ..... **Error! Bookmark not defined.**

ABSTRACT

This assignment conveys how the development of a interface viewer tool, like ifconfig on Linux, in python. The tool retrieves network configuration details such as the hostname, platform and network interface data. My solution to this problem was to use Python's modules these were argparse, socket, and pexpect. The main functionalities of these modules are to modularize the ifconfig data retrieval, TCP-based server and client connection communitication and detailed TCP flows using Wireshark. The results of the assignment showcase my ability to use these modules as used in this years curriculum.

INTRODUCTION

The Objective:

The objective was to code a simplified interface tool that:

1. retrieves and displays network configuration details on a local machine.

2. implemtation of a client-server architecture for the transmission and receiving of network interface data.

3. Demonstrate TCP communication through practical analysis of its flows.

The problem:

The problem does emphasize platform compatibility but unfortunately I was unable to tackle this problem as I took my time learning and understanding how the objective should be reached and formulated a plan to first get it working on linux. This report outlines the design, implementation, testing and analysis of my solution meeting the assignment requirements.

Figure 1: The Problem

DESIGN

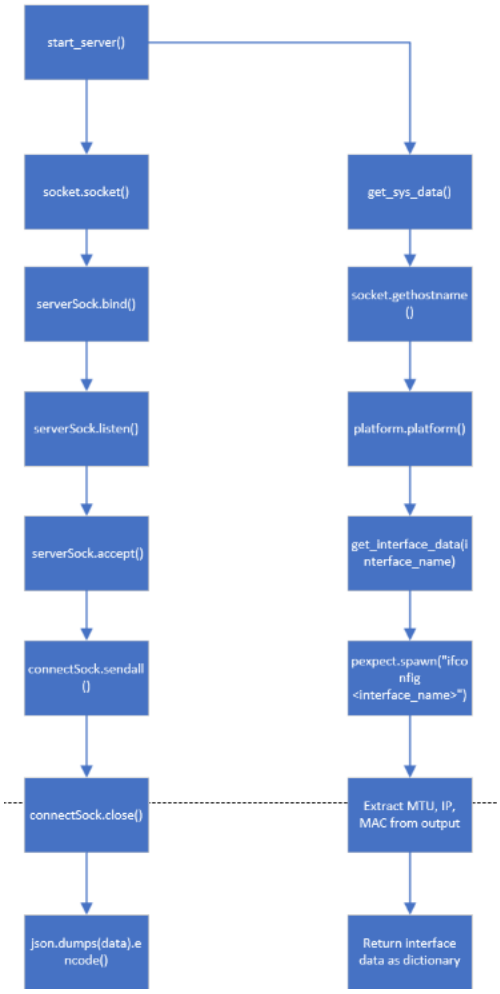
High Level Design: Structure Chart:

The system consists of two main components:

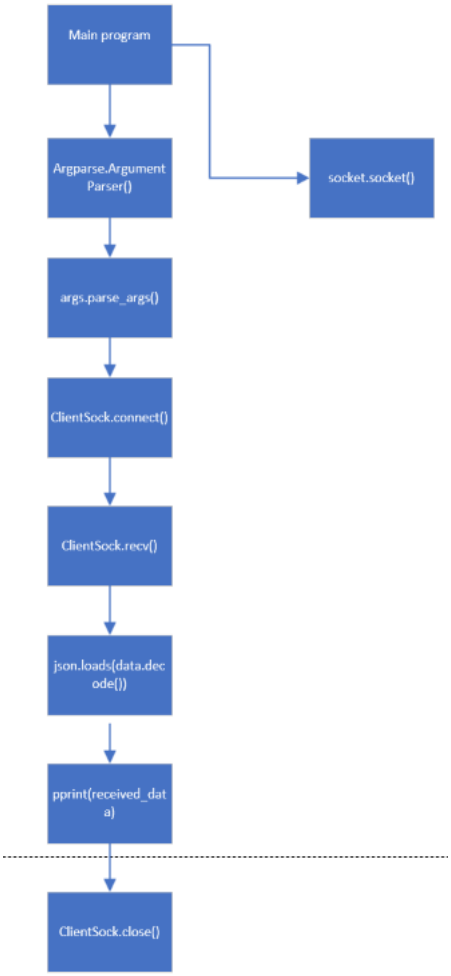
1. Local data retrieval module: A python script that gethers the network interface details and formats them as a dictionary

2. Client-server Communication: implements a TCP-based interaction where the server transmits data to the client upin the client request

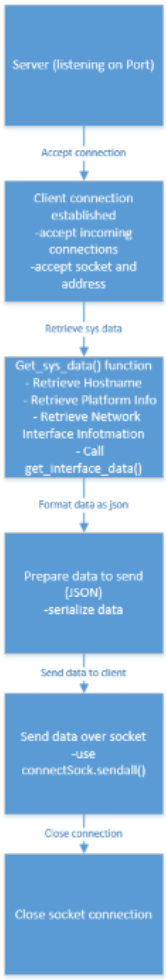
Server structure chart:



Client structure chart:



Block diagram for server:



1. SERVER INITIALIZER – THE SERVER LISTENS FOR INCOMING CONNECTIONS ON A SPECIFIED PORT(USING SERVERSOCK.BIND() AND SERVERSOCK.LISTEN()

2. client connection – when a client connects, the server then accepts the connection and retrieves the clients socket connection and address

3. data collection – the server then to gather system and network data by calling get_sys_data() function, this involves:

- Getting the system hostname (socket.gethostname())
- Retrieving the system platform (platform.platform())
- Gathering interface details (mtu,ip,mac) by calling the get_interface_data() function for each interface

4.Data serialization – once the system data is gathered it is formatted as a dictionary which is then serialized with json.dumps()

5. data transmission – The serialized JSON data is sent to the connected client using connectSock.sendall()

6. connection close – finally, after sending the data the server closes the connection using connectSock.close() to clean up

CLIENT BLOCK DIAGRAM



1. CLI argument parsing – parses the CLI argument using argparse.parse_args() to get the server IP and port, the client will connect to this address

2. TCP connection setup – creates a socket and establishes a connection to the server using the server's IP and port clientSock.connect()

3. Sending request – when the connection is established the client wasits for data from the server, the client then sends a request or connections to the server

4. Receive data – the client waits to receive data from the server using clientSock.recv(), the server sends the data in JSON format

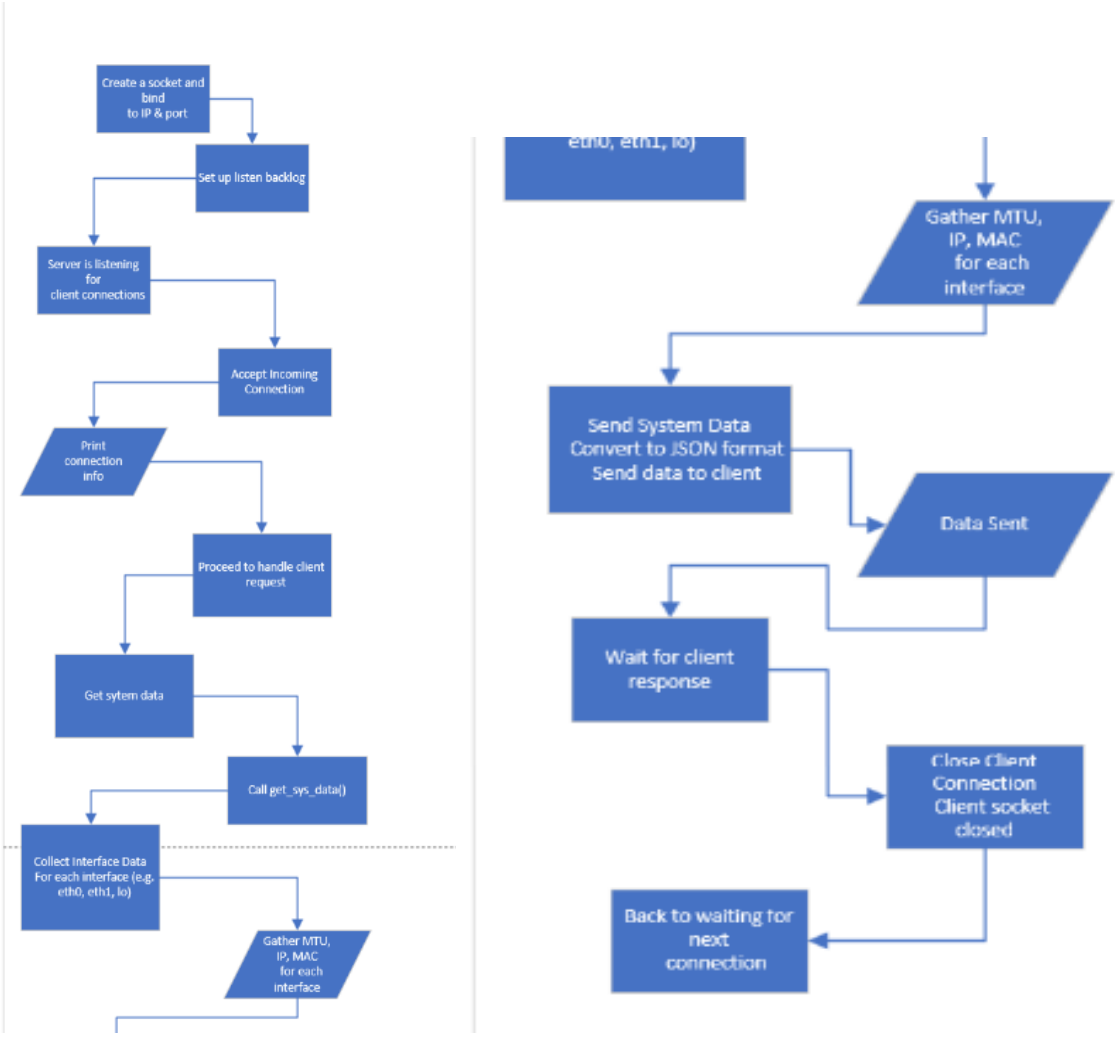
5. Data deserialization – the received data which is in JSON format then is decoded and deserialzied using json.loads() converting it to a python dictionary for easier access and processing

6. Displaying data – the client pretty prints(pprint) the data displaying key values (hostname, platform and interfaces)

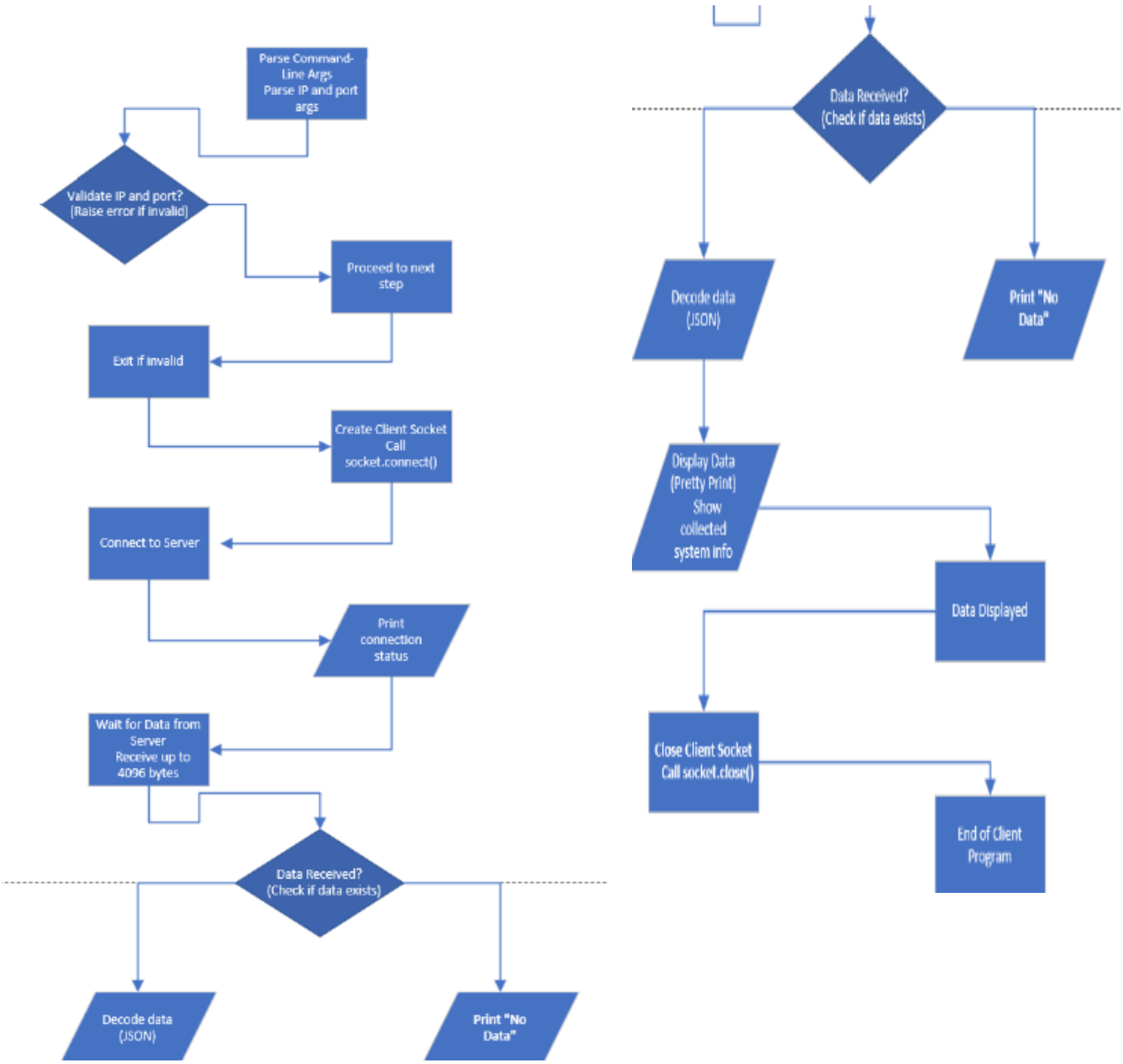
7. Socket closure – finally, after the data is displayed, the client closes the socket connection using clientSock.close()

Low Level Design: Algorithm:

Server Flowchart



Please refer to the appendix for a further breakdown.



Please refer to the appendix for a further breakdown.

IMPLEMENTING THE PROGRAM

To begin I researched the code I knew I would need to begin this assignment from past lab exercises and the helpful links provided by the lecturer

The first module I began exploring was how to return the host name, for this I found gethostname() to be sufficient in returning a string containing the host name of where the python code was running, I also took note of the fact that it doesn't always return a fully qualified domain name and I could use getfqdn() for that.

The next necessary step was to find a function that prints what platform the code was currently running on, for this I used a link provided that was for the platform module, the syntax is as follows: print(" ", platform.platform) I also noted that the return of this function would tell me the exact platforms OS name, version and other additional information, if I wanted to only return the platforms name I could use platform.system() .

Next on my agenda was to figure out how to write dictionary data out to a file in JSON, I found these methods below:

1. Json.dump(data) – used for converting python data to a JSON string

2. Json.loads(s) – JSON-encoded string to python data

3. Json.dump(data,outfile) – used to write python data out to a JSON string and then into a file

I knew I would need at least two of these later on down the line and thought it was best practice to note it as I went through my own notes.

I decided to use the pexpect module to spawn a subprocess that is similar to a command line interaction instead of the subprocess module as I thought using a module I have used before would give me a lead in starting this function in the code due to familarety, I began noting how pexpect is used to simulate the user typing on the command line in which in turn creates a child process.I noted I would only need four lines of code for the pexpect module although it is considered legacy to new python updates I was using python 3.4.3 so I knew it would not have affected me. I noted the four syntaxs I would need:

1. Pexpect.spawn – for spawning a subprocess and controlling it

2. Expect() - to stop the child process with a time stamp of End of File or TIMEOUT

3. Child.before – used on the text printed, up to be expected response pattern

4. .decode() – used to decode the data that would have printed on the command line had an actual user typed the command., lastly I noted I would decode this data to utf-8

Unfortunately for myself, I began to note how I will extract the data I needed from the pexpect decoded data, I decided to look into splitting the lines of data line by line and then indexing into each individual line to find the string I was looking, for example ("MTU"), I decided I was going to :

1. Use splitlines() to break the ifconfig command into separate lines

2. Split the first line with (line[0].split) by itself based on whitespaces

3. Index into the first line and pull the mtu and so on for the next interface value

Unfortunately, again, I researched on an easy way to obtain the data for the MAC address I stumbled upon the get_mac_address module and took a few notes on how I might implement this into my code with it only being two lines I was grateful I found this solution.

Lastly for part one I resurfaced old notes on how to save all this data into a dictionary, after doing a quick refresher and taking down new examples I decided to take on part 1.

I took my time and broke the code down into very specific tasks with comments and the first snippet goes as follows:

Problems with my first iteration of task 1:

The main two problems I had whilst testing and running this code was that I was hardcoding the the splitting of the lines and indexing to where the values I was searching for always had to be on the same line and the other problem I faced was with the get_mac_address module, however I did come to solving these problems but it was no quick fix.

The first error I got when running was that the get_mac module could not be found, after doing some research and testing on three other virtual machines and adapting the code to work on windows for a while I could not find a solution. It took two days to admit defeat and start looking for other options, I decided to try splitting and indexing it out but of course that was another problem I had, the hardcoding. To fix the hardcoding problem I had asked my class mates what they had done to retrieve the pexpect data and they recommended to use regular expressions. This fixed my hardcoding issues and the get_mac issues, I took this into account when going into part two of the assignment. In hindsight I should've used this from the start but blindsighted by wanting to get my code to work I yielded and moved on to part two with this new knowledge.

Please refer to the appendix for the full code breakdown.

Fixed part one

Part 2:

Using the past research I had done with the pexpect method, importing JSON and how to get the platform name and hostname and the new knowledge of using regular expressions I went into part two confidentl.

I began my preparations for part two by returning to the socket module, in which we had been doing in our lab for a number of weeks at that point so it was fresh in my mind, I noted what I would need such as:

1. Socket.socket(socket.AF_INET, SOCK_STREAM) – this creates the tcp socket for the server

2. socketAddr = ("0.0.0.0", port) – to set the specified host or port number. On top of this you can set the host and port number as a global variable/

3. serverSock.bind(socketAddrt) – bind the socket to the specified host and port number.

4. serverSock.listen(1) – begin listening for incoming connections with a backlog specified

5. serverSock.accept – accept socket for client connections

6. Additionally I was recommended to dynamically search for the names of the interfaces, I found socket.if_nameindex() returns the names of the interfaces into a tuple.

Next I look into how to obtain the name of the clients host machine, found two methods:

1. Client_address - gives the remote address but is provided during the connection establishment process

2. Getpeername() – is directly linked to the socket and can be useful if I lost the client_address although this function causes more overhead it is safer to use this function.

Lastly I needed to understand and research regular expressions, I took to our lecture notes and noted the important pieces of information I needed to use in my code

1. Re,search – returns none if the regex pattern is not found in the string, if the pattern is found, the search returns a match object

2. MTU_match.group(1) – if the search returns a match object the digits in group one will be captured and written out to a string, the groups will be in the regex expression symbolized into another set of brackets. Capturing groups allows me to isolate and retrieve specific parts of the match found that I would want for example a MAC address.

The last research I need is what to actually use for the regex line, for this I used regex 101 website to configure to correct sentences to look for , for each item I am looking for.

Please refer to the appendix for the full code breakdown.

TESTING

Present clearly the test results for each part of the assignment. Test cases should include valid and invalid input data as well as test of all boundary cases. Screen captures of results and Wireshark flow graphs may be used.

didn't work
get_mac

```
#!/usr/bin/env python3
import pexpect
import platform
import socket
#import get_mac to retrieve mac address
from get_mac import get_mac_address
#import pretty print from the pprint module
from pprint import pprint
def main():

# task 1 retrieve the hostname of the device and prints it
        hostname = socket.gethostname()
        #print the host name to the user
        print("host's name is ",hostname)


# task 2 retrieves the operating system version the code is running on using
        #the platform module
        SystemPlatform = print("You are on the platform:", platform.platform())
```

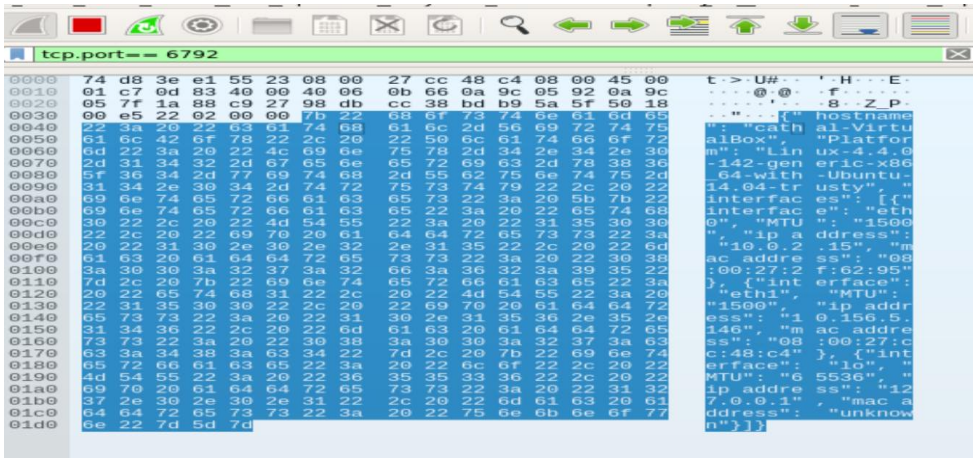
USING socket.if_nameindex() for finding the interface names instead of harding coding

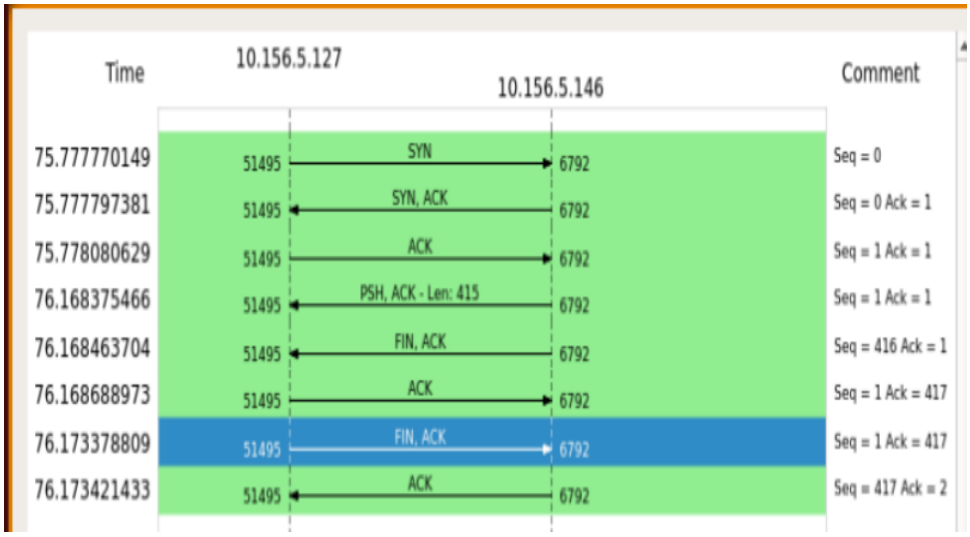
like this

```
#specify what interface the info is comi
interfaces = ["eth0","eth1", "lo"]
```

use this :

```
▸ Frame 19: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
▾ Ethernet II, Src: PcsCompu_cc:48:c4 (08:00:27:cc:48:c4), Dst: 74:d8:3e:e1:55:23 (74:d8:3e:e
  ▾ Destination: 74:d8:3e:e1:55:23 (74:d8:3e:e1:55:23)
      Address: 74:d8:3e:e1:55:23 (74:d8:3e:e1:55:23)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: PcsCompu_cc:48:c4 (08:00:27:cc:48:c4)
      Address: PcsCompu_cc:48:c4 (08:00:27:cc:48:c4)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
▸ Internet Protocol Version 4, Src: 10.156.5.146, Dst: 10.156.5.127
▾ Transmission Control Protocol, Src Port: 6792, Dst Port: 51495, Seq: 417, Ack: 2, Len: 0
    Source Port: 6792
    Destination Port: 51495
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 417    (relative sequence number)
    [Next sequence number: 417    (relative sequence number)]
    Acknowledgment number: 2    (relative ack number)
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x010 (ACK)
    Window size value: 229
    [Calculated window size: 29312]
    [Window size scaling factor: 128]
    Checksum: 0x2c29 [unverified]
    [Checksum Status: Unverified]
    Urgent pointer: 0
  ▸ [SEQ/ACK analysis]
```





ANALYSIS

Comments on the implementation details and on the test results, pointing out any limitations of your code and your TCP protocol analysis.

All exceptions and unexpected results should be noted. If some part of the work was unsuccessful, or gave apparently wrong results this should be clearly shown, together with measures or alternative actions which were taken, or could reasonably be taken, to overcome these problems.

```
#!/usr/bin/env python3
import pexpect
import platform
import socket
#import get_mac to retrieve mac address
from get_mac import get_mac_address
#import pretty print from the pprint module
from pprint import pprint
def main():

# task 1 retrieve the hostname of the device and prints it
        hostname = socket.gethostname()
        #print the host name to the user
        print("host's name is ",hostname)


# task 2 retrieves the operating system version the code is running on using
        #the platform module
        SystemPlatform = print("You are on the platform:", platform.platform())
```

Dynamically finding the interface names results:

By using:

```
#dynamically find interface names and return as a list of tuples
interfaces = [iface[1] for iface in socket.if_nameindex()]
```

```
C:\temp\programming\programming>python tcpclient1.py --ip 192.168.0.235 --port 6793
Connected to server at 192.168.0.235:6793
data recieved from server
{'Platform': 'Linux-4.4.0-142-generic-x86_64-with-Ubuntu-14.04-trusty',
 'hostname': 'cathal-VirtualBox',
 'interfaces': [{'MTU': '65536',
                 'interface': 'lo',
                 'ip address': '127.0.0.1',
                 'mac address': 'unknown'},
                {'MTU': '1500',
                 'interface': 'eth0',
                 'ip address': '10.0.2.15',
                 'mac address': '08:00:27:2f:62:95'},
                {'MTU': '1500',
                 'interface': 'eth1',
                 'ip address': '192.168.0.235',
                 'mac address': '08:00:27:cc:48:c4'}]}
```

The tcp flows and what to talk about them:

CONCLUSIONS

Where the requirnments met?

Were the results found expected?

Could the results be extended or used elsewhere?

What improvements could be made?

REFERENCES

https://docs.python.org/3/library/socket.html

https://www.geeksforgeeks.org/platform-module-in-python/

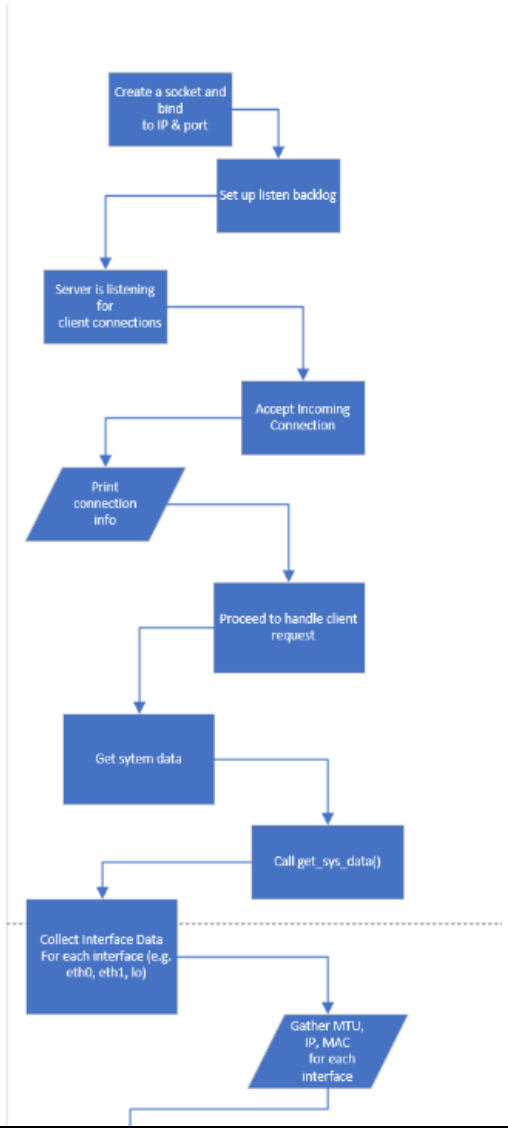
https://www.geeksforgeeks.org/how-to-use-python-pexpect-to-automate-linux-commands/

https://docs.python.org/3/howto/argparse.html

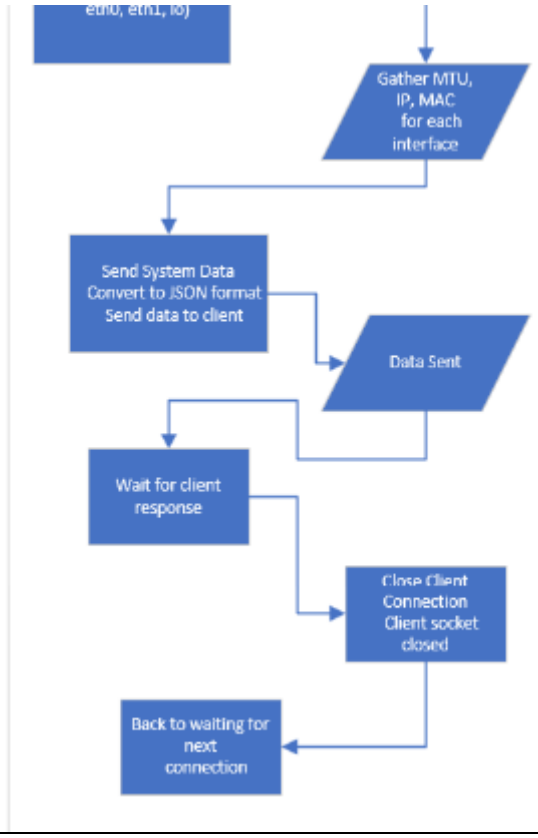
APPENDIX A

FLOWCHARTS/PSEUDOCODE

Server Flowchart and explaination

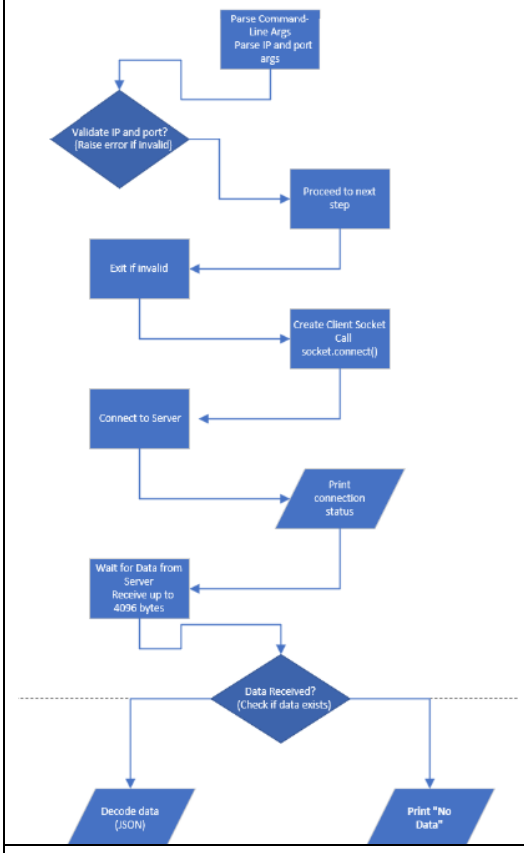


- The server initializes a socket and binds it to a specific ip and port
- Configures the maximum number of queued clinets connections the server can accept
- Indication that the server is now waiting for oncoming traffic
- Acception from the server from the client
- Logs details about the connected client
- Server prepares to respond to the clients request
- Server begins the process of gathering data
- Call get_sys_data
- The server iterates through interfaces
- Collects detailed information

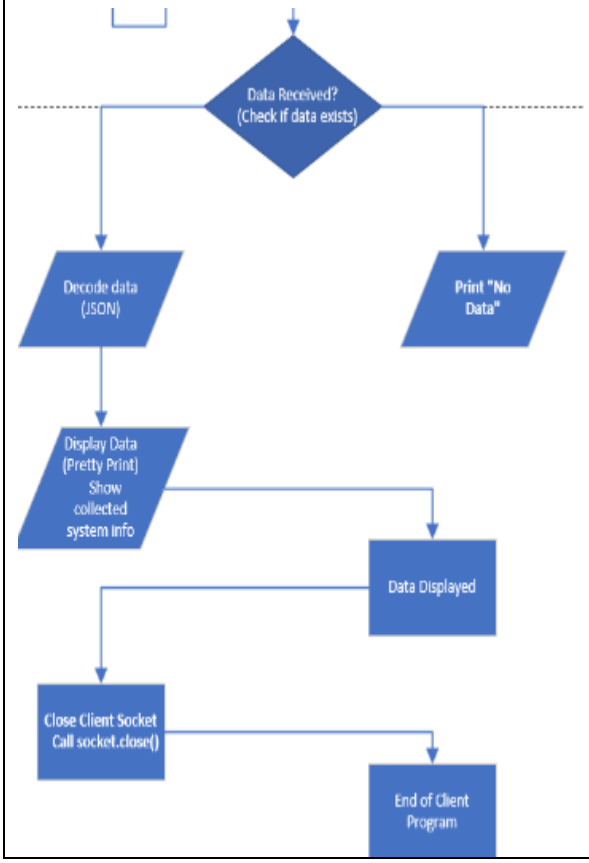


- **Sends the system data and converts to JSON**
- Wait for clients acknowledgemnts
- The server closes the connection
- Server returns to listening mode

CLIENT FLOWCHART:



- **Extracts input arguments**
- Ensures that the provided ip and port are valid
- Initializes a socket object
- Connects to the server
- Logs if the connection was successful or not
- Waits to receive data from the server



- **DECODES THE RECEIVED DATA**
- formats and displays the system information
- closes the socket connection

APPENDIX B

PYTHON CODE LISTINGS

Part 1:

Code:

```
#!/usr/bin/env python3
import pexpect
import platform
import socket
#import get_mac to retrieve mac address
from get_mac import get_mac_address
#import pretty print from the pprint module
from pprint import pprint
def main():

# task 1 retrieve the hostname of the device and prints it
        hostname = socket.gethostname()
        #print the host name to the user
        print("host's name is ",hostname)


# task 2 retrieves the operating system version the code is running on using
        #the platform module
        SystemPlatform = print("You are on the platform:", platform.platform())
```

Explanation:

1. The shebang was to ensure that python3 was being used when running the script, past knowledge from previous labs I attended

2. Imported all the nesacary modules I noted I would need

3. I began by retrieving the host name using the gethostname() function to return the name of the host and print it

4. I then retrieved the operation system version of the platform the code is running on using platform.platform() and following this printing it

```
# task 3 Retrieve the interface data using pexpect

        #start child process using spawn
        child = pexpect.spawn("ifconfig enp0s3")
        #stop child process when at end of file using expect
        child.expect([pexpect.EOF, pexpect.TIMEOUT])
        #return the child process as a byte string using before
        OutputBytes = child.before
        #decode the data to UTF-8
        print(OutputBytes.decode('utf8'))
```

5. The next task was to retrieve the interface data of the host machine, using the notes and examples I researched I had the correct syntax to be able to run the ifconfig command through the pexpect module and decoding the returned data in the process

```
# task 3.1 retrieve first interface (enp0s3)
        #name the first interface and print to the user
        intf1 = "enpos3"
        print("the first interface is",intf1)
        #split the interface lines
        lines = OutputBytes.splitlines()
        #split just the first line
        LineZero = line[0].split
        #index into this line and pull mtu + print
        print("the" LineZero[2] "value is" LineZero[3])
        #store the mtu value
        mtuOne = Linezero[3]
        #split the second line by its whitespaces and index in
        LineOne = Line[1].split
        #print the ip value
        print("the" LineOne[1] "ip is" LineOne[1])
        #store the ip
        Ipv4One = LineOne[1]

        #use the get_mac_address to find mac addr
        MacOne = get_mac_address(interface = intf1)
        #print the mac
        print("the make address is", MacOne)
```

6. The next step was to retrieve the interface data from the returned output of the pexpect module

7. By splitting the lines ,line by line, I could index into the individual lines and pick out the string I wanted and printing in the process. This is bad practice I came to learn because I assumed that the output of the ifconfig would return in the same format each time and I am esstially hardcoding the indexing to find search for something that may not be on that line. Before having this knowledge I continued on to split and index in until I had the mtu and ip address

8. I used the get_mac_address module to find the mac address and print it as I come to realize this was the longest mistake to fix.

9. Lastly I do run this for all interfaces on my machine.

```
# Task 4 save in a dictionary

        mydict = {
                "Hostname" : hostname,
                "platform" : SystemPlatform,
                "Interface One" : intf1,
                "MTU One" : MtuOne,
                "Ip address One" : Ipv4One,
                "MAC One" : MacOne,
                "Interface Two" : intf2,
                "MTU Two" : MtuTwo,
                "Ip address Two" : Ipv4Two,
                "MAC One" : MacTwo,
                 }
```

10. I then continue on to save all informational data into a dictionary

```
#task 5 display using ppretty print for user friendliness

        pprint(mydict)


# Task 6 Writing the dictionary data out to a file in json format

        with open("interface.json", "w") as fout:
            mydict = json.dump(mydict, fout)

if __name__ == "__main__":
        main()
```

11. I then pretty print the dictionary for ease of use and because it is easier for the user to read

12. Lastly write the dictionary out to a python file

Part 2:

Code:

ifinfo.py X | tcpclient1.py X | *tcpclient.py X | *get_interface_data.py X

```
#!/usr/bin/env python3
#server
import socket
import pexpect
import platform
import json
import re
from pprint import pprint
host = "0.0.0.0"
port = 1578

def get_interface_data(interface_name): #define func that uses the parsed intf
names
        try:
                #start a child process to run a ifconfig
                child = pexpect.spawn("ifconfig {}".format(interface_name))
                #stop the child process when you are the end of file
                child.expect([pexpect.EOF, pexpect.TIMEOUT])
                #return the process as a byte string using before then decode
                OutputBytes = child.before.decode('utf-8')
```

explanation:

1. Importation of the modules needed to complete the assignment

- Socket – handle TCP socket creation
- Platform – retrieve system information (OS version and version)
- Json – enables the serialization of python dictionaries for communication
- Pexpect – allows execution of shell commands(ifconfig)
- Re – facilitates regular expressions

2. Spawn a subprocess using pexpect and control it. Simulation of the user typing into the CLI.
    - .expect - Wait for the child process to finish at the specified time(end of file)
    - List comprehension to see all data before the match and decode.

```
#step 2 extract the mtu value
                #re.search, searches and returns a match object
                mtu_match  = re.search(r'\bMTU:(\d+)',OutputBytes)
                #variable that stores the digits found in regex
                mtu = mtu_match.group(1) if mtu_match else "unknown"

#step 3 extract the IP addresses for the given interfaces
                ip_match = re.search(r'inet\s+addr:(\d+\.\d+\.\d+\.\d
+)',OutputBytes)
                ip_address = ip_match.group(1) if ip_match else "unknown"

#step 4 extract the mac address for the given interfaces
                mac_match = re.search(r'HWaddr\s+([0-9a-fA-F:]+)',OutputBytes)

                mac_address = mac_match.group(1) if mac_match else "unknown"

                #store the name of interface+mtu+ip address as key value pairs
                return {"interface":interface_name,
                        "MTU": mtu,
                        "ip address": ip_address,
                        "mac address": mac_address
                        }
```

1. Extracts the values of the given interfaces using regex re.search function
    - Re.search if the pattern set is found the method returns a match object in brackets that will return the actual text found
    - If the re.search function returns a matched object, .group will return the first group of the regular expression that is isolated by brackets.
    - This is done for all information needed, mtu, mac address and ip address. Finally error handling is used to print “unknown” if the match object is not returned
2. This information is then returned into a dictionary.

```
                    }
#step 5 error handling
                # catch a error that might happen in try block, print an error
                #message , if a dictionary returns it should show unknown
        except Exception as e:
                print("Error retrieving data for {}: {}".format(interface_name,
e))
                return { "interface" : interface_name,
                "MTU": "unknown",
                "ip address": "unknown",
                "mac address":"unknown"
                }
```

3. This part of the code is an error handler for the entire try block, if regex were to crash or become corrupt. In this case the dictionary returned will not have interface information but just list this expected information as unknown.

```
def get_sys_data():             #gathering the sys data
        #retrieve the systems hostname
        hostname = socket.gethostname()
        #retrieve detailed info about the system OS
        system_platform = platform.platform()
        #dynamically find interface names and return as a list of tuples
        interfaces = [iface[1] for iface in socket.if_nameindex()]
        #loop through the list comprehension of interfaces and collect data
        interface_data = [get_interface_data(interface) for interface in
interfaces]
        #list comprehension that creates a new list tjat contains the
interfaces list info


        #return the interfaces info as a dict
        return {"hostname":hostname,
                "Platform":system_platform,
                "interfaces":interface_data,
                }
```

1. This function gathers the system data that the regex cannot.
    - Retrieves the system hostname using the socket module
    - Retrieves the systems platform information(linux/windows)
    - Dynamically searches for the name of the interfaces of the machine using the socket module and returns into a list(eth0,enp0s3)
    - Finally all interface system information is returned as a dictionary

```
def start_server():     #start the server

        #creating a server socket for client connections
        serverSock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        #Bind the socket to the localhost and port
        serverSock.bind((host,port))            #2-tuple
#task 1
        #start listening for incoming connections with a backlog of 1
        serverSock.listen(1)


        print("Server is listening on {}:{}".format(host, port))
```

1. This function starts a server socket for server-client connection by:
    - Creates a socket for TCP socket connection
    - bind the socket to a specified host and port
    - start listening for incoming connections with a backlog of one
    - print the name of the host and port that server is listening to, for the user

```
        #forever loop
        while True:
                #accept a socket for client connection
                connectSock, addr = serverSock.accept()
                # the connection is established with the client
                print("Connection established with {}".format(addr))
                try:
                        #assign the function that gets sys data to the variable
data
                        data = get_sys_data()
                        #pretty print the data collected by the function
                        pprint("System Data Collected: {}".format(data))
                        #send all the data from the get_sys func +encode
+convert to json
                        connectSock.sendall(json.dumps(data).encode('utf-8'))
                except Exception as e:
                        print("Error during client communication: {}".format(e))
                finally:#close the socket
                        connectSock.close()
                #error handling if the data is not sent

if __name__ == "__main__":
        start server()
```

1. create a forever loop
2. accept a socket for client connection
3. print to which client address the server is connected to
4. parse the data from the get_sys_function
5. pretty print the data found for user-friendliness
6. serialize the data to a JSON format and use sendall to the client
7. error handling to catch any exception within the try block.
8. Finally close the server-client connection

TCP client side:

```
#!/usr/bin/env python3
import argparse # command line option and arugment parsing libary
import socket
import json
from pprint import pprint

def main():
        #step 1 create a parse object to handle cli arguments
        parser = argparse.ArgumentParser()
        # step 2 add the argument for the server IP address
        parser.add_argument("--ip", help="Server IP address",required=True)
        #step 3 add the argument for the server port number
        parser.add_argument("--port", help="Server Port number",
type=int,required=True)
        #step 4 parse the ariguments provided on the command line
        args = parser.parse_args()
```

1. Import the modules needed to fulfll the assignment requirnments.
    - Argparse – CLI option to parse arguments by the user
    - Socket – to handle the TCP socket creation and connection
    - JSON – enable serialization of python dictionaries for communication
2. Create a parse object for handling command line arguments
    - Define the required arguments to specify the servers port number + ip address. Ensure the type of data allowed in this case integers, “help = ‘ ’ “ describes the value of what argument is being parsed
    - Parse the arguments provided by the user

	and store them in the args object

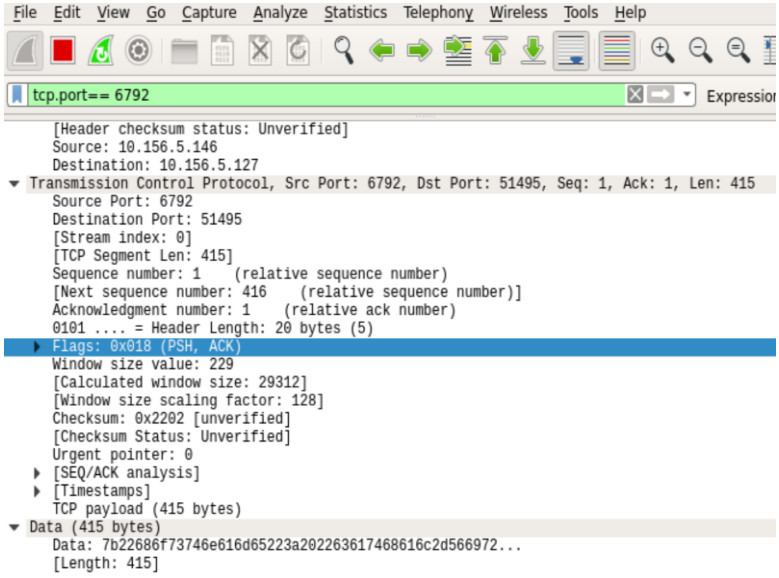
`#step 5 create a client socket for communication`
`ClientSock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
`#step 6 connect the client socket to the server IP and port provid`
`ClientSock.connect((args.ip, args.port))`
`#step 7 print a message confirming that the connection to the serv`
`print("Connected to server at {}:{}".format(args.ip, args.port))`
	1. Creation of the client socket for communication with the server
	
	2. Connect to the servers specific port number and IP and print to the user the specified port and IP

`print("Connected to server at {}:{}".format(args.ip, args.port))`
`try:`
`# Receive data from the server`
`data = ClientSock.recv(4096)`
`# Buffer size is 4096 bytes`
`#step 8 check if the data is recieved from the server`
`if data :`
`#step 9 decode the recieved data to a string`
`#step 10 load the JSON data into a python dict`
`recieved_data = json.loads(data.decode('utf-8'))`
`#step 11 print a message to confirm data is recieved from server`
`print("data recieved from server ")`
`#step 12 pretty print to display the dict in user friendly way`
`pprint(recieved_data)`
`# step 13 Parse and pretty-print the JSON data`
`else:`
`#step 14 print a message if no data recieved`
`print("no data recieved from server")`
	1. Try block for receiving data from the server
	
	• Waits to receive up to 4096 bytes of data from the server
	
	2. Check the data that was received from the server
	
	• Decode the data to a string and load the JSON string to a python dictionary
	
	• Pretty print the dictionary results and a error handling message if no data is received

`print("no data recieved from server")`
`#step 15 error message if theres an issue while recieving data`
`except Exception as e:`
`print("Error receiving data: {}".format(e))`
`finally:`
`#step 16 ensure the connection is terminated after operation`
`ClientSock.close()`
 	
`ime__ == "__main__":`
`.n()`
	1. Exception error for catching any problems in the try block.
	
	2. Ensure the connection is terminated between the client and server after all operations are complete

APPENDIX C

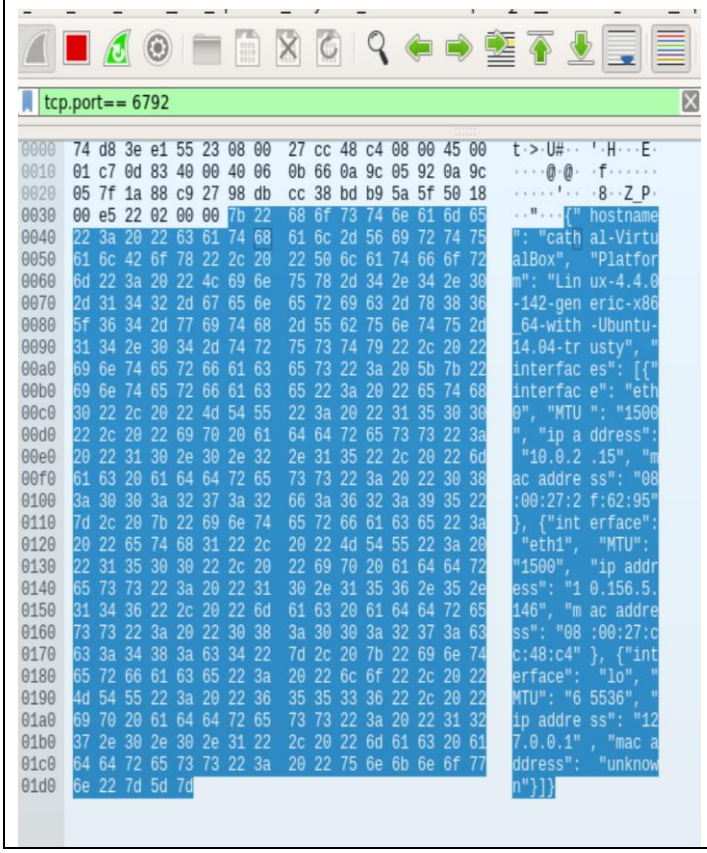
WIRESHARK CAPTURES



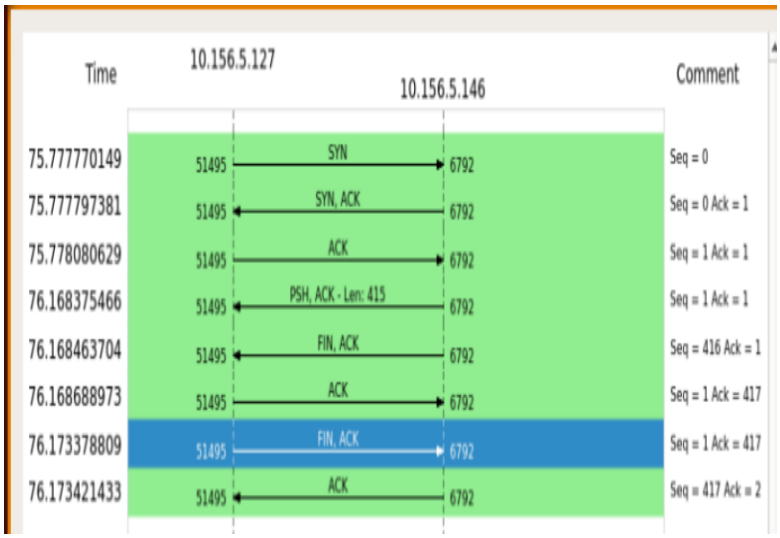
This packet specified displays the flag PSH and ACK, TCP flags, signalling that the sender wants the data to be immediately transmitted to the receiver whilst acknowloding the past transmission

The source port is the specified port number shown in the server

The destination port is dynamically allocated port used by the client



In this wireshark image it provides a detailed view of the raw application payload carried in the TCP connection. The hex and ASCII data reveal a structured JSON response, the dictionary with all the data such as the interface name, host names, platform mtu, MAC addresses, and Ip addresses



1. The client starts the connection by sending a request
2. The server acknowledges the request and responds
3. The client confirms the connection establishment
4. The client sends data to the server
5. Ther server begins termination of the connection
6. The client acknowledges the termination request
7. The client sends a final termination request
8. The server accepts